

Parliament Insights API

Systems Analysis and Design of a Generative AI and Backend Layer over
Validated Research Data

Software Architecture Document (SAD) describing the requirements, architecture, data model, and API design of a system that consumes, without modifying them, the scientific artifacts produced by the undergraduate thesis *Parliament Graph Architecture*.

Felipe Echeverria Vilhalva

B.Sc. in Computer Science — UEMS

September 25, 2026

Contents

1	Introduction	2
1.1	Context	2
1.2	Objectives	2
1.3	Scope	2
1.4	Target Audience	2
2	Requirements	3
2.1	Functional Requirements	3
2.2	Non-Functional Requirements	3
3	Use Case Analysis	5
3.1	Use Case Diagram	5
3.2	Specification of the Main Use Cases	5
4	Software Architecture	7
4.1	Layered Overview	7
4.2	Layer Descriptions	7
4.3	Architectural Patterns and Decisions	8
5	Data Model	9
5.1	Entity-Relationship Diagram	10
5.2	Data Dictionary (summary)	11
6	Main Flows (Sequence Diagrams)	12
6.1	Flow 1 – Structured Query (UC01)	12
6.2	Flow 2 – RAG Search (UC02)	12
6.3	Flow 3 – Batch Summarization (UC03)	13
7	API Design	14
7.1	REST Endpoints	14
7.2	Contract Example – POST /v1/query	14
8	Technology Stack	15
9	Infrastructure and HPC-Readiness	16
10	Testing Strategy	17
11	Security Considerations	18
12	Roadmap and Future Work	19
A	Appendix A – Architecture Decision Record (ADR-01)	20

1 Introduction

1.1 Context

The *Parliament Graph Architecture* project (undergraduate thesis) produced a reproducible pipeline for analyzing parliamentary co-authorship networks, statistically validated against null models (hypothesis H1, $p < 0.005$ across four consecutive years). The thesis is intentionally closed as a scientific artifact: its responsibility ends at generating auditable indicators (`analysis_{year}.json`, per-deputy metrics CSV, GEXF, and SQLite).

The **Parliament Insights API** is a *separate* system that **consumes** these artifacts as a read-only data source and adds a product layer: a traditional backend API (NestJS) combined with a Generative Artificial Intelligence layer (RAG, agents, and LLM-based summarization), without ever altering the thesis's statistical validation core.

1.2 Objectives

- Demonstrate Software Engineering competence in a system that integrates a traditional backend, AI agent orchestration, and vector-data infrastructure.
- Serve as technical portfolio material aligned with *Generative AI Data Scientist / Engineer* roles, covering Transformer architecture, RAG, agents, and HPC-readiness practices (containerization, queues, separation of compute layers).
- Preserve the methodological integrity of the thesis, treating it as an immutable source of truth.

1.3 Scope

This document covers: functional and non-functional requirements; use cases; layered architecture; data model; main flows (sequence diagrams); API design; technology stack; testing strategy; security and infrastructure considerations; and roadmap. It does not cover: the thesis's statistical methodology (already documented in its own README), nor implementation source code (this is an analysis-and-design document, prior to full coding).

1.4 Target Audience

Technical recruiters, portfolio reviewers, the academic advisor, and the author himself as an implementation reference.

2 Requirements

2.1 Functional Requirements

ID	Name	Description
FR01	Structured metric query	The system must allow querying, in natural language, already-validated structural metrics (density, modularity, centralities) filtered by year, returning the exact source value with no numeric generation by the LLM.
FR02	Semantic search over propositions	The system must allow searching legislative propositions by meaning similarity (not just keyword), with optional filtering by community (Louvain) and year.
FR03	Thematic community summary	The system must generate and expose a textual summary of the thematic pattern of a community of deputies, based on the summaries of propositions co-authored by its members.
FR04	Year-over-year comparison	The system must allow comparing the same indicator (e.g., modularity, density) across multiple years of the 2022–2025 period.
FR05	Intent routing	The system must automatically classify the intent of the user’s question (structured query, semantic search, or summary) and route it to the appropriate tool.
FR06	Source traceability	Any response involving LLM generation must cite the source propositions/documents used as context.
FR07	Summary caching	Already-generated community summaries must be cached and reused, avoiding a new LLM call for the same community/year.
FR08	Authentication and authorization	Access to the API must be authenticated (API key or JWT) with per-client rate limiting.
FR09	Thesis artifact ingestion	The system must periodically import the artifacts generated by the thesis pipeline (JSON/CSV/GEXF) into the operational database, without modifying them at the source.
FR10	Batch embedding processing	Generation of embeddings for proposition summaries must be executable asynchronously/in batch, decoupled from the API’s synchronous request path.

2.2 Non-Functional Requirements

ID	Category	Description
NFR01	Accuracy / Reliability	Structured query responses (FR01, FR04) must be deterministically correct with respect to the source data; the LLM acts only as a language-interpretation layer, never as the source of the numeric value.
NFR02	Performance	Structured queries must respond within 1s (p95); RAG queries within 5s (p95), including the LLM call.
NFR03	Scalability	The embedding-generation and summarization layer must be horizontally scalable via independent workers, allowing multiple instances with no shared in-memory state.
NFR04	Portability / HPC-readiness	All services must be runnable as containers (Docker), stateless at the compute layer, and compatible with batch execution, reflecting cluster scheduling principles (e.g., SLURM).
NFR05	Observability	Every LLM call must be logged with: prompt, model used, tokens consumed, and latency, for cost-auditing purposes.
NFR06	Security	External communication over HTTPS; secrets (LLM API keys) managed via environment variables/a secret manager, never committed to version control.
NFR07	Maintainability	Strict layer separation (API / Orchestration / Tools / Data), following the same <i>Dependency Rule</i> principle already adopted in the thesis.
NFR08	Testability	Automated test coverage $\geq 80\%$ in the API and tools layers, with integration tests covering the three main flows (FR01–FR03).
NFR09	Cost	The architecture must allow switching between a local LLM (Ollama) and a paid LLM API, enabling per-query cost comparison.
NFR10	Source data integrity	The system must never write to or alter the thesis's original artifacts; all ingestion is read-only at the source.

3 Use Case Analysis

3.1 Use Case Diagram

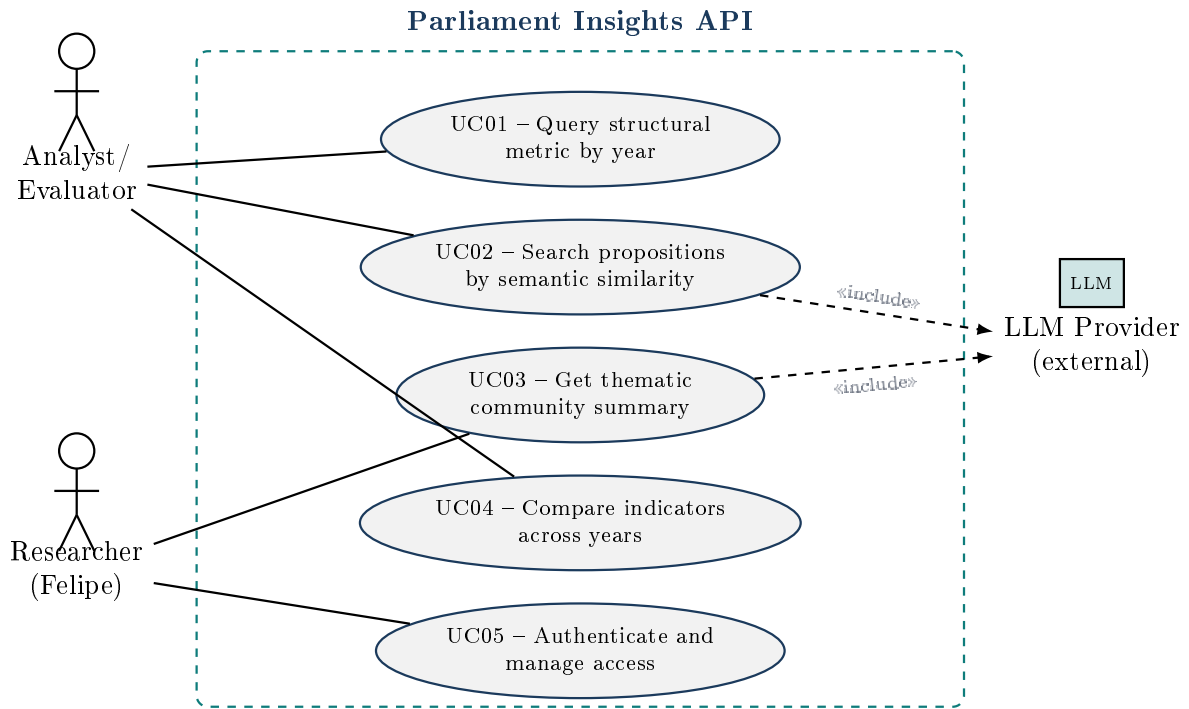


Figure 1: Use cases of the Parliament Insights API

3.2 Specification of the Main Use Cases

UC01 – Query a structural metric by year

Primary actor

Analyst/Evaluator

Precondition

Thesis artifacts already ingested into the operational base (FR09).

Main flow 1. User submits a natural-language question (e.g., *"what was the modularity in 2024?"*).

2. The agent classifies the intent as a structured query (FR05).

3. The *Structured Query Tool* extracts parameters (year, metric) and runs a parameterized query against the database.

4. The result is formatted in natural language and returned with the exact value.

Alternative flow

Metric or year not found → an explicit "data not available" response, with no inference.

Postcondition

Correct response, traceable back to the source `analysis_{year}.json`.

UC02 – Search propositions by semantic similarity

Primary actor

Analyst/Evaluator

Precondition

Embeddings of proposition summaries already generated (FR10).

Main flow 1. User asks a thematic question (e.g., *"which environmental proposals did community 3 co-author in 2025?"*).

2. The agent identifies semantic-search intent and extracts filters (community, year).
3. The *RAG Tool* embeds the question and runs a similarity search (pgvector) with a metadata filter.
4. The retrieved context is sent to the LLM, which generates an answer citing the source propositions (FR06).

Postcondition

Response with explicit citation of the `proposition_ids` used.

UC03 – Get a thematic community summary

Primary actor

Researcher

Main flow 1. An (asynchronous) batch process collects the summaries of all propositions co-authored by a Louvain community.

2. The LLM generates a thematic summary of the set.
3. The summary is stored in a cache (`CommunitySummary`).
4. The API exposes the ready-made summary on demand, with no new LLM call (FR07).

Constraint

The summary is explicitly labeled as an exploratory tool, not as part of the thesis's statistical validation.

4 Software Architecture

4.1 Layered Overview

The architecture follows the same *Dependency Rule* principle adopted in the thesis: each layer depends only on the layer immediately below it, never the reverse, and the crossing between technologies (Node.js $\leftrightarrow$ Python) happens at a single, well-defined boundary (API $\rightarrow$ Orchestration).

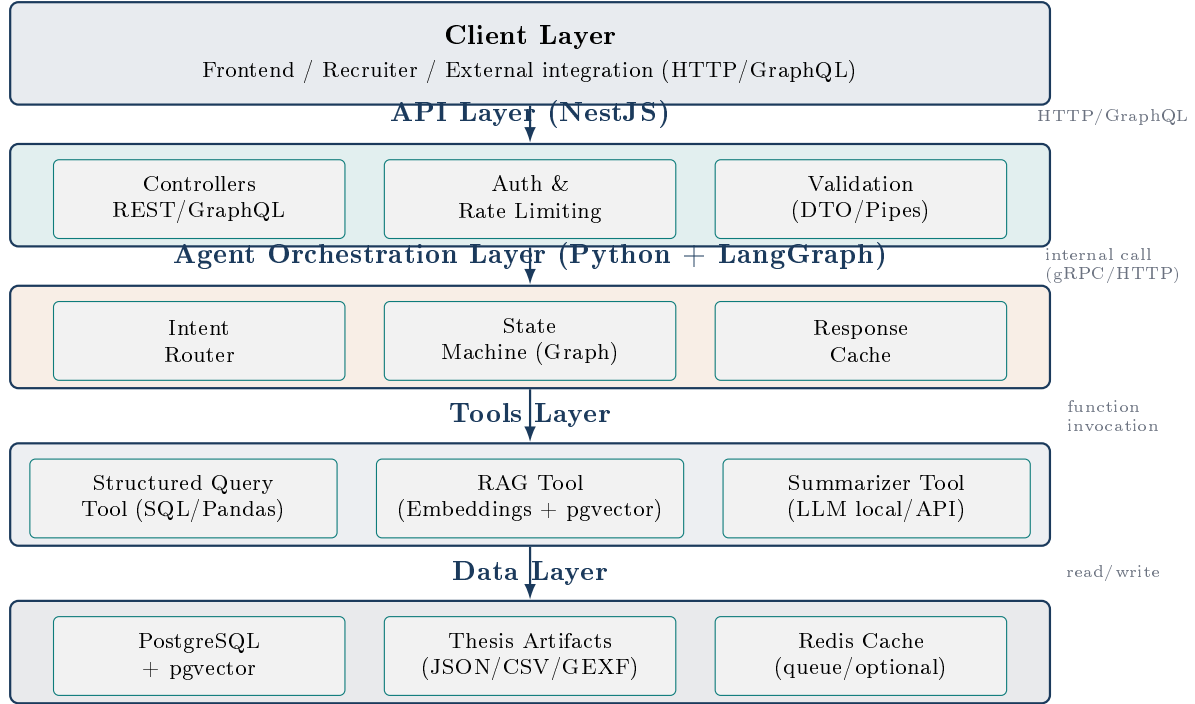


Figure 2: Layered architecture of the Parliament Insights API

4.2 Layer Descriptions

Layer	Responsibility
API (NestJS)	HTTP/GraphQL exposure, authentication, input validation (DTOs), rate limiting, and translation between the public API contract and the internal call to the orchestration service. Contains no AI logic.
Agent Orchestration (Python + LangGraph)	A state machine representing the agent's flow: receives the question, classifies the intent (FR05), decides which tool(s) to invoke, and aggregates the final result. Does not access the database directly – delegates to the tools.
Tools	Three independent, individually testable implementations: the <i>Structured Query Tool</i> (deterministic), the <i>RAG Tool</i> (semantic search + generation), and the <i>Summarizer Tool</i> (batch summarization). Each tool encapsulates a single responsibility.
Data	PostgreSQL with the <code>pgvector</code> extension for embeddings; thesis artifacts as a read-only import source; Redis as an optional queue for asynchronous processing.

4.3 Architectural Patterns and Decisions

- **Monorepo with clear module boundaries** (`api/`, `agent/`, `data/`) instead of multiple repositories – see ADR-01 (Appendix A).
- **Separation between numeric-value generation and language generation**: the *Structured Query Tool* never delegates to the LLM the computation of a number already validated by the thesis – this avoids statistical hallucination (NFR01).
- **GraphRAG**: semantic search (RAG Tool) always combines vector similarity *with* a structural filter over graph metadata (community, year), rather than pure vector search.
- **Decoupled batch processing** for summarization (FR10), preparing the system for the cluster-execution pattern (a job queue analogous to SLURM) required by AI/HPC roles.
- **LLM provider independence**: the Tools layer abstracts the provider (local Ollama or a paid API) behind a common interface, enabling cost/quality comparison (NFR09).

5 Data Model

5.1 Entity-Relationship Diagram

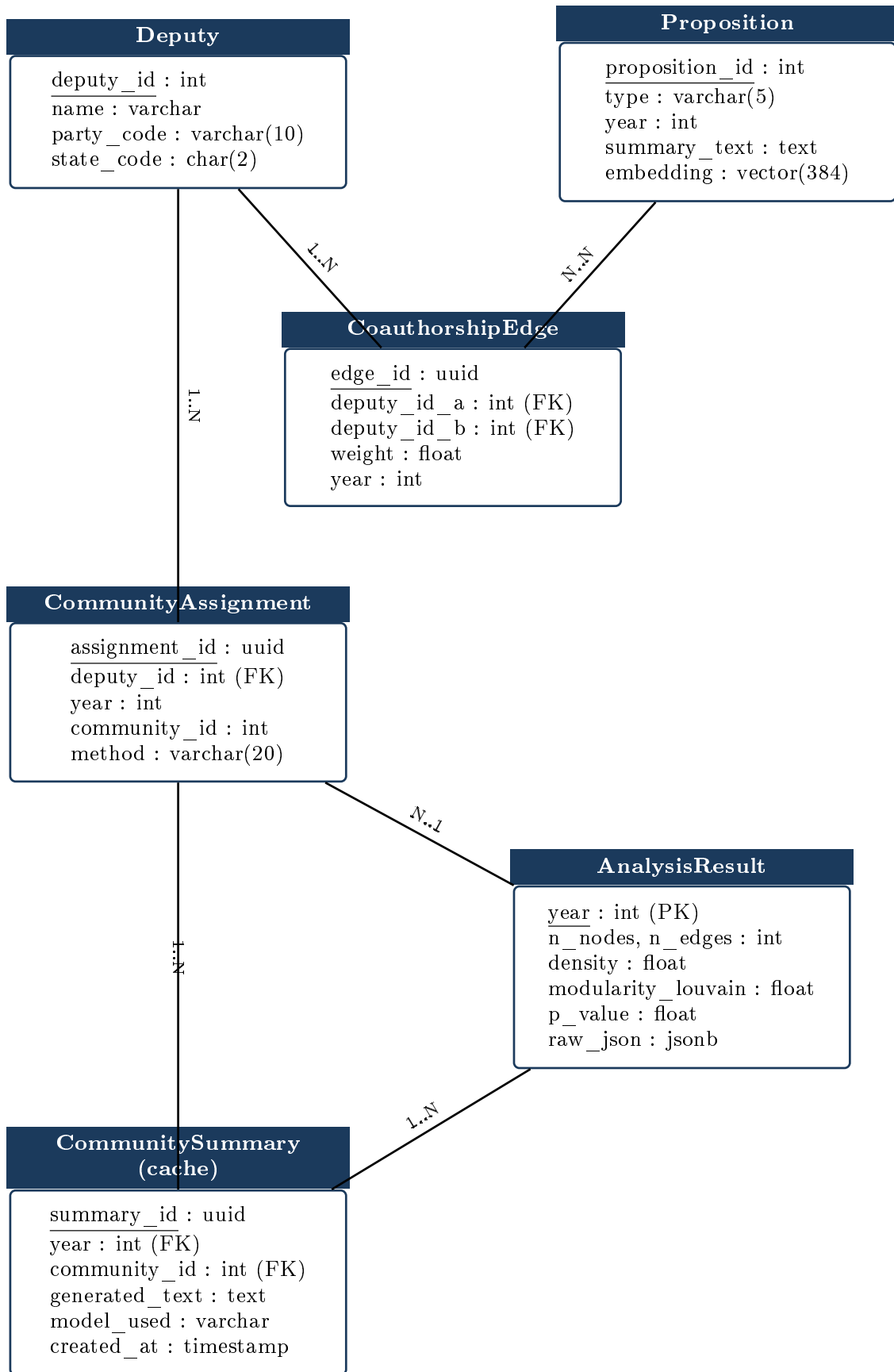


Figure 3: Operational data model of the Parliament Insights API

5.2 Data Dictionary (summary)

Entity	Source	Notes
Deputy	Imported from the thesis (metrics CSV)	Natural key <code>deputy_id</code> preserved from the original Chamber data.
Proposition	Imported + enriched	The <code>embedding</code> field (384-dimension vector, <code>sentence-transformers</code>) is generated at this layer; it does not exist in the thesis.
CoauthorshipEdge	Imported from the GEXF	Normalized replica of the co-authorship graph's edges, for efficient relational querying.
CommunityAssignment	Imported from the metrics CSV	Louvain/Label Propagation community assignment per deputy/year.
AnalysisResult	Imported from <code>analysis_{year}.json</code>	Append-only table; never rewritten, only versioned by year – guarantees NFR10.
CommunitySummary	Generated at this layer	LLM result cache; includes <code>model_used</code> for cost auditing (NFR05).

6 Main Flows (Sequence Diagrams)

6.1 Flow 1 – Structured Query (UC01)

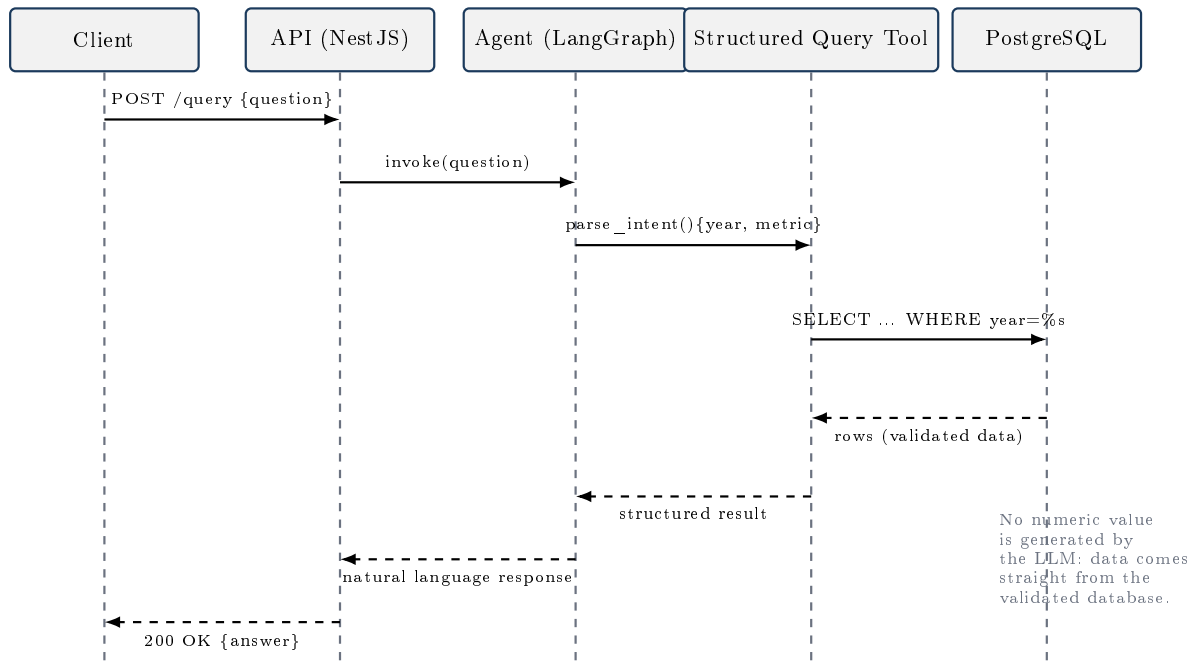


Figure 4: Execution sequence of a structured query

6.2 Flow 2 – RAG Search (UC02)

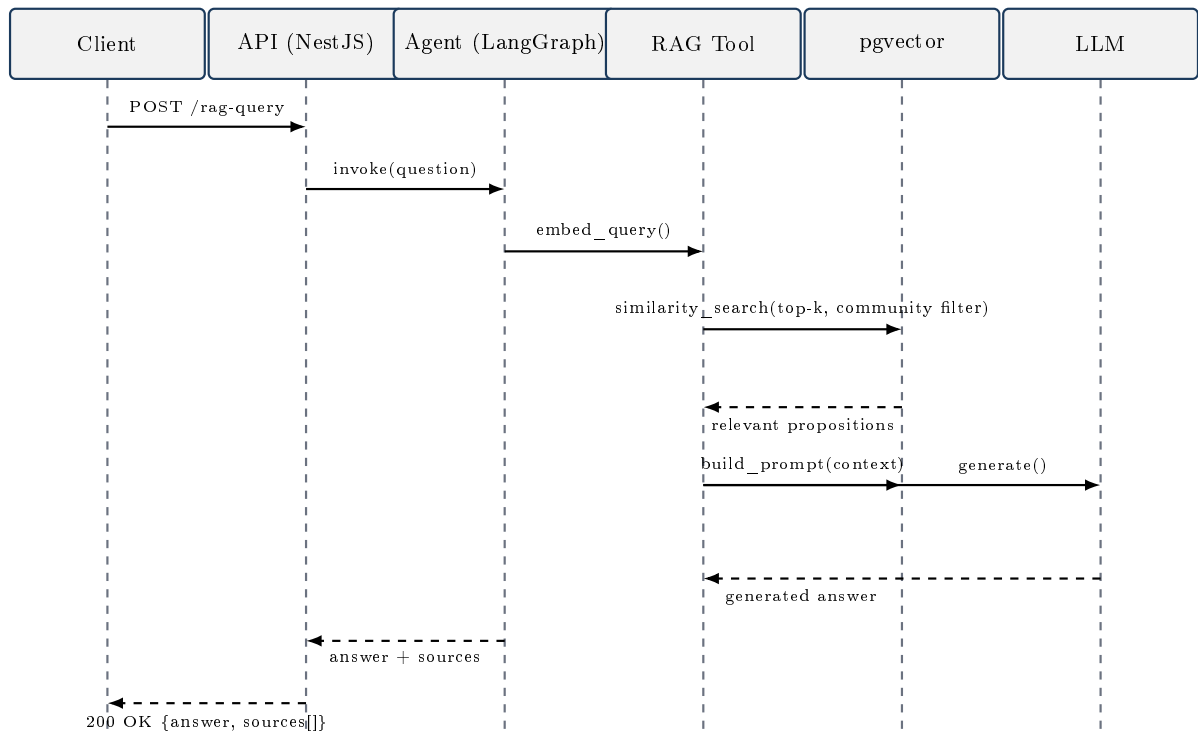


Figure 5: Execution sequence of a semantic search with augmented generation

6.3 Flow 3 – Batch Summarization (UC03)

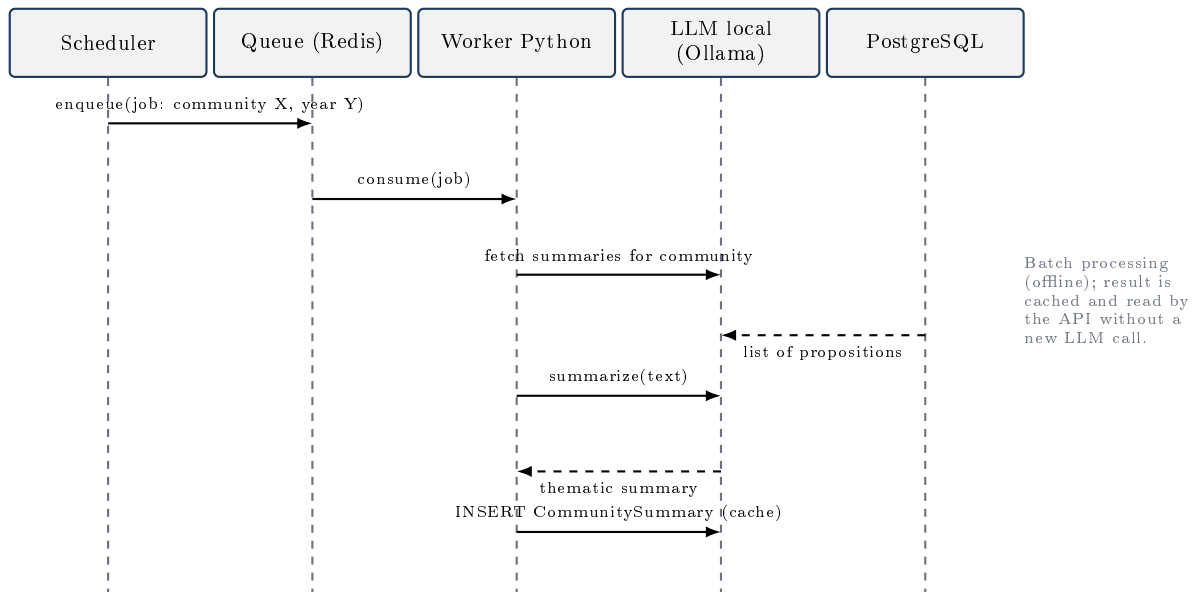


Figure 6: Asynchronous execution sequence for the community summary

7 API Design

7.1 REST Endpoints

Method	Route	Description
POST	/v1/query	Single entry point for the agent; routes to FR01, FR02, or FR03 depending on the classified intent.
GET	/v1/metrics/{year}	Direct structured query (bypassing the agent), returns the raw AnalysisResult for the year.
GET	/v1/metrics/compare?years=2022,2025&metric=modularity	Year-over-year comparison (FR04).
POST	/v1/search	Direct semantic search (FR02) without text generation – returns only the documents.
GET	/v1/communities/{year}/{id}/summary	Retrieves an already-cached community summary (FR03/FR07); 202 Accepted if still processing.
POST	/v1/admin/ingest	Triggers ingestion of the thesis artifacts (FR09); restricted to an administrative role.

7.2 Contract Example – POST /v1/query

Request:

```
{
  "question": "What was the network's modularity in 2024?",
  "context": { "preferred_language": "en-US" }
}
```

Response (200):

```
{
  "intent": "structured_query",
  "answer": "In 2024, the observed (Louvain) modularity was 0.634, with p-value 0.005 (significant).",
  "source": {
    "type": "AnalysisResult",
    "year": 2024,
    "field": "louvain.modularity"
  },
  "generated_by_llm": false,
  "latency_ms": 340
}
```

Response (200) – RAG intent:

```
{
  "intent": "semantic_search",
  "answer": "Community 3 co-authored 4 environmental propositions in 2025...",
  "sources": [
    { "proposition_id": 88213, "type": "PL", "year": 2025, "score": 0.87 },
    { "proposition_id": 88540, "type": "PEC", "year": 2025, "score": 0.81 }
  ],
  "generated_by_llm": true,
  "model_used": "llama3.2:3b",
  "latency_ms": 2130
}
```

8 Technology Stack

Layer	Technology	Rationale
HTTP/GraphQL API	NestJS (TypeScript)	Stack already mastered by the author; strong typing and modules aligned with the layering principle.
Agent orchestration	Python + LangGraph	Standard market ecosystem for orchestrating agents with explicit state (decision graph).
Embeddings	sentence-transformers	Local execution, no API cost, full control over vector dimensionality.
Vector store	PostgreSQL + pgvector	Reuses the same relational DBMS already in use, avoiding a dedicated vector database.
Local LLM	Ollama (Llama 3.2 3B)	Enables cost/quality comparison against a paid API (NFR09) and demonstrates local model-deployment expertise.
Queue / async processing	Redis + BullMQ	Decouples embedding generation/summarization from the API's synchronous path (FR10).
Containerization	Docker + Docker Compose	Same approach as the thesis; a direct path to HPC-readiness (NFR04).

9 Infrastructure and HPC-Readiness

Although this project's scope does not require a real HPC cluster, the architecture is designed to reflect the principles that make a system *HPC-ready*, addressing the "nice to have" SLURM/containers requirement of the reference job posting:

- **Stateless compute layer:** the embedding/summarization *worker* keeps no in-memory state between runs – each job is independent, analogous to a SLURM *job array*.
- **Batch-first:** embedding generation runs in batch over all pending propositions, not one at a time on demand.
- **Containerization:** every service runs in an isolated Docker image; migrating to Apptainer/Singularity (common in academic HPC environments) would only require swapping the container runtime, not the code.
- **Queue as a scheduler abstraction:** the Redis/BullMQ queue fulfills, at a smaller scale, the same role a SLURM scheduler fulfills in a cluster – distributing jobs across available workers.

10 Testing Strategy

Level	Approach
Unit	<i>Structured Query Tool</i> : 100% of query-parameter combinations tested against an in-memory database/fixture. <i>RAG Tool</i> : mocked embedding provider and vector store.
Integration	Full API → Agent → Tool → Database flow, for the three main use cases (UC01–UC03), with a test database populated from a fixed sample of the thesis artifacts.
Contract	Request/response schema validation for each endpoint (FR08) via automated contract tests.
Generation quality	For FR02/FR03, manual sample-based evaluation (not automatable in CI) of retrieved-document relevance and of the generated summary's fidelity to the source summaries.
Cost regression	Tests that log tokens/latency per LLM call, alerting if the average per query exceeds a defined threshold (NFR05/NFR09).

11 Security Considerations

- External LLM provider API keys are never committed to version control – managed via environment variables/`.env` (following the same pattern already used in the thesis with `pydantic-settings`).
- Per-API-key rate limiting to mitigate abusive use and contain the cost of external LLM calls.
- Input sanitization before composing prompts, mitigating prompt injection via proposition text or user questions.
- The ingestion endpoint (`/v1/admin/ingest`) is restricted to an administrative role and never publicly exposed.
- Source data (the thesis) is treated as *read-only* throughout the ingestion pipeline – no reverse writes are allowed (NFR10).

12 Roadmap and Future Work

1. **V1 (MVP):** *Structured Query Tool* + thesis artifact ingestion + `/v1/metrics` endpoint.
2. **V2:** RAG over proposition summaries + `/v1/search` and `/v1/query` endpoints.
3. **V3:** Batch community summarization + cache + `/v1/communities` endpoint.
4. **V4 (extension):** Lightweight fine-tuning (LoRA/PEFT) of a small model specialized in the legislative domain, comparing quality/cost against generic Ollama – a direct connection point to the local-LLM consultancy idea.
5. **V5 (extension):** Extraction of the `agent/` module as a reusable Python package, should other graph projects arise that could benefit from the same RAG/agent layer (see Section 6.3).

A Appendix A – Architecture Decision Record (ADR-01)

Field	Content
Title	Monorepo vs. multiple repositories for the API/Agent/Data layers
Status	Accepted
Context	The project spans two languages (TypeScript/Python) and four logical layers. It is an individual portfolio project, not a multi-team system.
Decision	Adopt a monorepo with api/ , agent/ , data/ folders, communicating via internal HTTP, instead of separate repositories per layer.
Alternatives considered	(1) Poly-repo with one repository per layer; (2) Monorepo with integrated <i>workspaces</i> (npm/uv).
Positive consequences	Unified reading of the architecture by an external reviewer; less cross-repository versioning overhead; single deployment via docker-compose .
Negative consequences	Weaker per-layer ownership isolation (irrelevant for a single-author context); should the agent/ module need to become a reusable library (Section 10, item V5), it will be extracted into its own repository at that point.
